

1. Explore repository

1) Model

공식 Repository의 `complete_verifier/models/` 디렉토리를 살펴보면, ResNet 계열의 CIFAR, MNIST 기반 모델들을 중심으로 한 다양한 CNN 아키텍처 파일들이 포함되어 있다. MLP, ResNet, WRN 등을 지원하며, 아키텍처에 존재하는 Custom operation를 바인딩하여 검증에 사용할 수도 있도록 하고 있다. 또한 Non-linear activation function인 sigmoid, tanh, gelu를 지원하며, Transformer와 self-attention에 대한 검증도 가능하다. 이외에도 README.md에 따르면 LSTM, Transposed convolution 등을 지원한다.

2) Verification config

`complete_verifier/exp_config/` 디렉토리를 살펴보면, 다양한 모델 아키텍처에 상응하는 검증 설정이 담긴 YAML 파일들이 있다. 각 파일은 모델 명세와 데이터셋 정의뿐만 아니라 attack의 종류 및 설정과 사용할 solver의 종류, Branch and Bound(bab)의 hyperparameter 설정 등을 담고 있다.

3) Comparison to Marabou

Marabou는 .nnet, .h5 등의 포맷을 중심으로 지원했지만, α, β -Crown은 .pkl과 .pth(pytorch 가중치 포맷)등을 기본 포맷으로 한다. Marabou는 SMT 기반 solver로 작동하기에 부등식 형태의 제약 조건들이 픽셀별로 전부 쿼리에 담긴 형태를 Verification config로 사용했지만, Bound propagation과 BaB 기반의 α, β -Crown은 간단한 파라미터 조정 정도만 담겨있는 간단한 형태를 사용한다. 지원하는 모델의 종류는 크게 다르지 않지만, α, β -Crown이 훨씬 더 방대한 모델을 검증할 수 있다고도 알려져 있다.

2. Running α, β -Crown

1) Model and Dataset

본 실험에서는 MNIST 숫자 이미지(28x28)를 분류하기 위한 SmallCNN 구조를 설계하고 학습했다. 이상적인 비교를 위해 이전 실험인 Marabou 검증에 사용했던 아키텍처를 그대로 재사용하였으며, ONNX 파일 포맷 파싱 문제로 정확히 같은 모델을 사용하지 못하고 같은 환경에서 재학습한 모델로 실험하였다.

- 상세 구조

Conv layer 1. Conv2d(1, 16, kernel_size=3, padding=1), ReLU(), MaxPool2d(2)

Conv layer 2. Conv2d(16, 32, kernel_size=3, padding=1), ReLU(), MaxPool2d(2)

FC layer 1. Flatten(), Linear(in_features=1568, out_features=64), ReLU()

FC layer 2. Linear(in_features=64, out_features=10)

- Accuracy: 98.71% (Marabou 검증에 활용한 모델의 Accuracy: 98.80%)

검증에 사용한 데이터 또한 Marabou 검증에서 사용했던 것과 동일하게 MNIST Test set의 Index 0 이미지 한 장(Label: 7)을 사용하였다.

2) Verification Property

$\epsilon = 0.001$ L_∞ -norm 영역 내의 모든 sample들에 대하여, 정답 Label인 7번의 출력 logit이 나머지 9개 class에서의 logit보다 항상 큼을 보장해야 한다. 이때, 모델이 정규화된 MNIST를 기반으로 훈련되었으므로, config YAML 파일에 mean: [0.1307], std: [0.3081]을 포함하여 입력을 정규화 공간으로 매핑한 후 검증한다.

3) Result

- Verified: safe-incomplete (PGD-100 attack failed)
- Verification time: 3.6264s
- CROWN Lower Bounds = [22.748, 16.235, 14.577, 12.749, 18.766, 18.509, 30.824, 19.828, 14.737]

3. Discussion

1) Result Interpretation

실행 로그 분석 결과, 정답 클래스와 오분류 클래스 간의 Initial CROWN Bounds가 위와 같이 모두 큰 양수의 값으로 계산되었다. 이는 $\epsilon = 0.001$ 이내에서 어떤 적대적 노이즈가 유입되더라도 정답 클래스가 오답 클래스들을 압도함을 수학적으로 증명한다. 특히 최종 결론 Branch-and-Bound 루프에 진입할 필요도 없이, 초기 단계에서 안전성을 찾아냈음을 의미한다. Marabou에서의 검증 결과가 UNSAT이었던 점과 같은 결과이다.

2) Comparison to Marabou

동일한 아키텍처와 학습 환경, 그리고 동일한 공격과 검증 조건에 대해 두 검증 도구를 비교한 결과는 아래 표와 같다. 가장 명확하게 드러난 차이점은 속도와 확장성이다. Marabou는 CPU 환경에서 UNSAT을 증명하기 위해 각 뉴런의 활성화 유무를 개별 제약식으로 쪼개어 Simplex 알고리즘으로 해결하므로, 크지 않은 CNN 모델에서도 한 번의 검증에 20분이 넘는 심각한 정체를 보였다. 반면, α, β -CROWN은 모든 뉴런의 의존 관계를 식으로 치환한 뒤 GPU 연산으로 일괄 전파하므로 매우 빠르게(3.62초) 검증을 완료하였다.

비교 기준	Marabou	α, β -CROWN
검증 결과	UNSAT	SAFE
소요 시간	20분 이상	3.6262초
연산 가속 장치	CPU	GPU (CUDA 가속)
사용 편의성	조건들을 부등식 형태로 픽셀별로 명시하는 등 스크립트 작업이 까다로움	구조화된 하나의 YAML 파일로 모델 경로, 데이터 전처리 및 다양한 하이퍼파라미터 제어
지원 기능 및 확장성	pytorch 기반 파일 포맷을 허용하지 않음 SMT 및 Simplex 기반 탐색	ONNX, pytorch 기반 파일 포맷 지원 CROWN, β -CROWN 등 강력한 기능 내장

표 1 Marabou와 α, β -CROWN의 비교

3) Pros and Cons of α, β -Crown

- Pros

GPU 기반의 압도적인 연산 속도: 대규모 행렬 병렬 연산 가속을 극대화하여 선형 완화 경계를 계산하므로 수백 배 이상의 턴어라운드 시간 단축을 달성할 수 있다.

다층적인 검증 파이프라인: 무조건 하드코딩된 solver를 돌리는 것이 아니라 PGD 공격 → CROWN → BaB 순으로 계층화된 검증 메커니즘을 사용하여, 본 실험과 같은 가벼운 명제는 BaB 연산 비용을 쓰지 않고 조기 종결시키는 효율성을 보여준다.

유연한 config 설정: 데이터 전처리 등 다양한 환경 조정을 자체 YAML 설정에서 일괄 지원한다.

- Cons

ONNX 호환성 이슈: ONNX 파일 포맷을 파싱할 때, onnx2pytorch 변환 모듈의 한계로 인해 필수가 아닌 속성(예: kernel_size)의 저장 여부 등에 따라 변환 에러가 발생한다.

복잡한 하이퍼파라미터 튜닝: YAML 내부에 존재하는 여러 알고리즘 파라미터(알파 학습률, 분지 선택 기준, 최적화 스텝 등)의 세팅에 따라 검증 바운드의 타이트함과 속도가 크게 좌우되므로 숙련도가 요구된다.